COMPLETE POSTMAN API TESTING NOTES

Negative Scenarios • Environment Variables • Response Chaining • JavaScript Assertions

Beginner-friendly explanation of the Flow(5).txt walkthrough
and the supplied Booking API.postman_collection.json

How to Use These Notes

These notes explain the exact topics shown in your two uploads: negative API scenarios, Postman environment variables, saving response data, reusing booking IDs and tokens, and writing JavaScript assertions. The exported Booking API collection is also inspected so the examples match the requests and scripts present in that file.

Source vs. added guidance The walkthrough and collection are the source material. Beginner explanations, improved scripts, additional test ideas, and best practices are added teaching guidance. Where the source leaves a scenario unfinished, the notes clearly describe it as a test design idea rather than a test already implemented.

Learning outcomes

- Create negative API tests for missing records and invalid request data.
- Create and select a Postman environment and use {{variableName}} syntax.
- Read a response body with pm.response.json() and save values with pm.environment.set().
- Reuse booking IDs and authentication tokens in later requests.
- Write assertions for status codes, response body, headers, and response time.
- Understand request order and run a repeatable booking lifecycle safely.

Contents

- 1. Quick view of the supplied Booking API collection
- 2. Negative testing fundamentals
- 3. Record not found - GET with invalid ID
- 4. Invalid characters and malformed JSON
- 5. Designing the other negative scenarios
- 6. Environment variables and {{fname}}
- 7. Saving a booking ID from a response
- 8. Reusing the booking ID in GET, PUT, and DELETE
- 9. Generating and reusing an authentication token
- 10. JavaScript test scripts and assertion patterns
- 11. Response-body assertions
- 12. Response-header and content-type assertions
- 13. Response-time assertions
- 14. Correct execution order for the collection
- 15. Real-world QA scenarios and test cases
- 16. Best practices, troubleshooting, interview revision, and exercises

1. Quick View of the Supplied Booking API Collection

The uploaded Postman collection is named Booking API. It contains two top-level folders: Positive Endpoints and Negative Scenario. The positive folder contains GET Booking Ids, GET Booking Details As Per Id, Create Booking, Update Booking, Token Generator, and Delete Booking. The negative folder contains GET Booking Details Invalid ID and Create Booking With Invalid Characters.

Request name	Method and route	Purpose
GET Booking Ids	GET /booking	Returns booking identifiers.
GET Booking Details As Per Id	GET /booking/{{b_id}}	Reads one booking using the saved ID.
Create Booking	POST /booking	Creates a booking and saves bookingid as b_id.
Update Booking	PUT /booking/{{b_id}}	Updates a booking using an authentication cookie.
Token Generator	POST /auth	Gets a token and saves it as b_token.
Delete Booking	DELETE /booking/{{b_id}}	Deletes the selected booking using the token cookie.
GET Booking Details Invalid ID	GET /booking/48746	Checks a not-found response.
Create Booking With Invalid Characters	POST /booking	Checks rejection of an invalid totalprice value.

Important collection finding The collection is not fully self-ordering. GET by ID uses {{b_id}}, and PUT/DELETE use both {{b_id}} and {{b_token}}. Those variables must exist before the dependent requests run. The recommended order is explained in Section 14.

2. Negative Testing Fundamentals

Positive testing checks whether an API works with valid inputs and valid conditions. Negative testing checks whether the API handles invalid inputs, missing information, unauthorized access, unsupported formats, and unavailable records correctly.

Why negative testing matters

- It verifies that invalid data is rejected instead of being saved silently.
- It checks that the API returns meaningful error status codes and messages.
- It helps prevent bad data from entering databases and downstream services.
- It checks that authentication and authorization rules are enforced.
- It verifies that the service remains predictable when clients make mistakes.

A negative test is not successful merely because the request fails. It passes when the actual behavior matches the expected behavior defined by the API contract. For example, an invalid booking ID should produce the documented not-found response, not an unrelated server error.

Test category	Example input	What to verify
Resource missing	Booking ID that does not exist	404 or the documented not-found response.
Invalid value	Text in a numeric field	Validation/parsing error and no booking created.
Missing field	firstname omitted	Documented required-field error.
Wrong format	Date like 31-99-2026	Documented date-format validation.
Too long	Text beyond the permitted maximum	Boundary validation according to the API schema.
Authentication	No token or invalid token	401/403 or API-specific authentication failure.

3. Record Not Found - GET with an Invalid ID

The walkthrough creates a request named GET Booking Details Invalid ID. It uses GET https://restful-booker.herokuapp.com/booking/48746 and expects HTTP 404 Not Found. The number 48746 is an example from the training exercise; use an ID that is confirmed not to exist in the test environment.

What happens step by step?

- The client sends a GET request for booking 48746.
- The server looks for that booking identifier.
- No matching booking is found.
- The server returns a not-found response, expected as 404 for this exercise.

```
GET https://restful-booker.herokuapp.com/booking/48746
```

Postman post-response test script

```
pm.test("Status code is 404", function () {
    pm.response.to.have.status(404);
});
```

Code explanation

Code	Meaning
pm.test(name, function)	Creates a named test shown in the Postman Test Results panel.
"Status code is 404"	Human-readable test name. It does not itself perform the check.
pm.response	Represents the response received for the current request.
to.have.status(404)	Asserts that the actual HTTP status is exactly 404.

If the API returns 200 for a missing record, the assertion fails. This is useful because returning 200 with an empty or misleading payload can hide a defect in the API contract.

Real-time example In an order-management system, a customer opens /orders/ORD-999999 after the order has been deleted or never existed. The API should return its documented not-found response, and the application should show an appropriate message rather than a blank order screen.

4. Invalid Characters and Malformed JSON

The walkthrough modifies the totalprice field and inserts special characters. The exported collection contains this body fragment: "totalprice": !@#$. Because the value is unquoted, the request body is not valid JSON. The expected response in the training collection is HTTP 400 Bad Request.

Invalid request body from the supplied collection (malformed JSON)

```
{
  "firstname": "Sneha",
  "lastname": "Karajagi",
  "totalprice": !@#$,
  "depositpaid": true,
  "bookingdates": {
    "checkin": "2016-04-22",
    "checkout": "2024-04-08"
  },
  "additionalneeds": "Breakfast and Meals"
}
```

Why does it fail?

JSON has strict syntax. A number must be a valid numeric literal, and a string must be enclosed in double quotes. The unquoted value !@#$ is neither a valid JSON number nor a valid JSON string. The server cannot parse the request body, so the demo exercise expects 400 Bad Request.

```
pm.test("Status code is 400", function () {
    pm.response.to.have.status(400);
});
```

Important distinction: malformed JSON vs. wrong field value

Test type	Example	What it tests
Malformed JSON	"totalprice": !@#$	JSON parser rejects the body before normal field validation.
Valid JSON, wrong type	"totalprice": "not-a-number"	API field validation/type checking.
Valid JSON, out-of-range	"totalprice": -10	Business-rule validation, if negative prices are prohibited.

QA improvement Keep these as separate test cases. If the purpose is to test field validation, keep the JSON syntactically valid and send a wrong type or invalid value. Otherwise, the test may only prove that the JSON parser rejects malformed syntax.

5. Designing the Other Negative Scenarios

Flow(5).txt lists three more scenarios but does not provide complete request bodies or expected status codes for them. The following are practical test designs. Confirm exact limits, required fields, formats, and status codes from the API documentation or contract before asserting them.

A. Values have too many characters

Purpose: verify maximum-length validation for text fields such as firstname, lastname, or additionalneeds.

- Find the maximum permitted length in the API specification. Do not invent a limit.
- Test one value at the maximum length.
- Test a value one character longer than the maximum.
- Verify whether the API rejects the request, truncates the value, or follows another documented behavior.

```
/* Example test idea only: replace 50 with the documented maximum. */
const overLimitName = "A".repeat(51);
```

B. Request sent with missing information

Purpose: verify required-field validation. Remove one required property at a time so the failure reason is clear. For example, remove firstname, remove bookingdates, or omit checkin. Do not remove several fields in one test unless the test specifically checks multiple missing fields.

```
{
  "lastname": "Rao",
  "totalprice": 250,
  "depositpaid": true,
  "bookingdates": {
    "checkin": "2027-01-10",
    "checkout": "2027-01-12"
  },
  "additionalneeds": "Breakfast"
}
```

In this example, firstname is intentionally omitted. Expected status and error response must come from the API contract; the supplied flow does not specify them.

C. Value is not in the correct format

Purpose: verify validation of dates, numeric values, booleans, or identifiers. Examples include a date in an unsupported format, a string in a numeric field, or a non-boolean value for depositpaid.

```
{
 "firstname": "Asha",
 "lastname": "Rao",
 "totalprice": "two hundred",
 "depositpaid": "yes",
 "bookingdates": {
  "checkin": "10-01-2027",
  "checkout": "12-01-2027"
 },
 "additionalneeds": "Breakfast"
}
```

Expected result is contract-dependent Do not assume every invalid request must return 400. Some APIs use 422 for semantically invalid content, 409 for conflicts, or a documented custom error. Assert the behavior specified by the API under test.

6. Environment Variables and {{fname}}

An environment is a named set of values for one target system, such as Development, QA, Staging, or Production. Environment variables let you reuse values without typing them into every request.

Create the Booking API environment

- Open the Environments area in Postman and create a new environment named Booking API.
- Add a variable with name fname.
- Set its value to Abhi, as shown in the walkthrough.
- Save the environment.
- Select Booking API as the active environment before sending the request.

```
Variable name: fname
Initial/current value: Abhi
```

In the Create Booking request body, replace a hard-coded first name with {{fname}}. Postman substitutes the active environment value when it sends the request.

```
{
 "firstname": "{{fname}}",
 "lastname": "Biradar",
 "totalprice": 874,
 "depositpaid": true,
 "bookingdates": {
  "checkin": "2027-01-10",
  "checkout": "2027-01-12"
 },
 "additionalneeds": "Breakfast and Meals"
}
```

What does {{fname}} mean?

The double curly braces tell Postman to resolve a variable. If the active environment contains fname = Abhi, Postman sends "firstname": "Abhi". The braces are Postman variable syntax; they are not JavaScript syntax.

Common variable scopes

Scope	Typical purpose	Example
Environment	Values for one target environment	baseUrl for QA; fname for this exercise
Collection	Shared values for requests in a collection	A common API base URL
Global	Broadly shared values across workspaces/collections	Use sparingly; can cause name collisions
Local / iteration data	Temporary or data-driven run values	A test row value in a collection run

Beginner check If {{fname}} appears literally in the request body or the request behaves as though it is missing, check that the variable name is spelled exactly as saved and that the correct environment is selected.

7. Save a Booking ID from the Response

When a POST request creates a booking, the API response includes a generated bookingid. The next requests need that identifier to retrieve, update, or delete the same booking. Saving the ID avoids hard-coding an ID that changes on every run.

Code from the walkthrough

```
var jsonData = pm.response.json();
pm.environment.set("b_id", jsonData.bookingid);
```

Line-by-line explanation

Code	Explanation
var jsonData	Declares a JavaScript variable named jsonData. var is valid, though const is usually preferred when the variable will not be reassigned.
pm.response.json()	Parses the response body as JSON and returns a JavaScript object.
jsonData.bookingid	Reads the bookingid property from the response object.
pm.environment.set("b_id", ...)	Stores the ID in the active Postman environment under the name b_id.

Example response (illustrative):

```
{
  "bookingid": 1505,
  "booking": {
    "firstname": "Abhi",
    "lastname": "Biradar",
    "totalprice": 874
  }
}
```

After the script runs successfully, the active environment should contain b_id with the returned ID. The value 1505 in the walkthrough is only an example; your server can generate a different ID.

Safer version for a repeatable test

```
const responseBody = pm.response.json();

pm.test("Create booking returned an ID", function () {
    pm.expect(responseBody).to.have.property("bookingid");
    pm.expect(responseBody.bookingid).to.be.a("number");
});

if (responseBody.bookingid) {
    pm.environment.set("b_id", String(responseBody.bookingid));
}
```

This version checks that the response contains bookingid before saving it. String(...) stores the identifier consistently as text, which is convenient when inserting it into a URL.

8. Reuse b_id in GET, PUT, and DELETE

After b_id has been saved, use it in every request that needs to address the same booking.

Request	URL pattern	Purpose
GET details	https://restful-booker.herokuapp.com/booking/{{b_id}}	Retrieve the booking created by POST.
PUT update	https://restful-booker.herokuapp.com/booking/{{b_id}}	Update that same booking.
DELETE	https://restful-booker.herokuapp.com/booking/{{b_id}}	Delete that same test booking.

Why this is better than hard-coding

Suppose the first run creates booking 1505 and the next run creates booking 1506. With a hard-coded URL ending in /1505, the second run might read or delete the wrong record. With {{b_id}}, Postman uses the ID created during the current run.

Real-time example An e-commerce test creates order 9021. It stores the order ID and uses it for GET order details, PUT shipping-address update, and DELETE/cancel order. The same chaining pattern applies to bookings, customers, invoices, and other resources.

9. Generate and Reuse an Authentication Token

The supplied collection has a Token Generator request: POST https://restful-booker.herokuapp.com/auth. Its request body uses the demo credentials shown in the collection. A successful response includes a token, which the script saves as b_token.

```
{
  "username": "admin",
  "password": "password123"
}
```

Token capture script from the walkthrough/collection

```
var jsonData = pm.response.json();
pm.environment.set("b_token", jsonData.token);
```

The first line parses the JSON response. The second reads the token property and stores it in the active environment under b_token.

Use the token for PUT and DELETE

The exported collection uses a Cookie header with the value token={{b_token}}. This means the HTTP header name is Cookie, and the cookie value is token= followed by the stored token.

```
Cookie: token={{b_token}}
```

Important correction to wording The walkthrough describes adding token={{b_token}} in the headers. The exported collection shows the exact configuration: header key Cookie and header value token={{b_token}}. Do not create a custom header named token unless the API documentation explicitly requires that.

Safer token capture script

```
const authBody = pm.response.json();

pm.test("Authentication returned a token", function () {
    pm.expect(authBody).to.have.property("token");
    pm.expect(authBody.token).to.be.a("string").and.not.empty;
});

if (authBody.token) {
    pm.environment.set("b_token", authBody.token);
}
```

The username and password in the uploaded material are public demo credentials for the practice API. Never reuse these credentials in another service. Do not store real passwords or long-lived tokens in a shared/public collection.

10. JavaScript Test Scripts and Assertions

A Postman test script runs after the server sends a response. In the current Postman interface, these scripts are commonly placed under Scripts → Post-response. The supplied export stores them as test events. The script checks whether the actual response matches an expectation.

Status-code assertion

```
pm.test("Verify Status Code is 200", function () {
    pm.response.to.have.status(200);
});
```

Part	Meaning
pm.test(...)	Registers a test.
"Verify Status Code is 200"	Name visible in Postman test results.
function () { ... }	Callback containing the assertion.
pm.response.to.have.status(200)	Passes only when the response status is 200.

When the request returns 200, the test passes. If the response is 201, 400, 404, or another status, this exact assertion fails. That does not necessarily mean the API is defective; it may mean the expected status in the test is wrong. The expectation must match the API contract.

Status codes to use in this exercise

Collection request	Expected status in supplied material	Notes
GET Booking Ids	200	Successful read.
GET Booking Details As Per Id	200	Successful read when ID exists.
Create Booking	200	This demo collection expects 200; many APIs use 201 for creation. Follow this

		API contract.
Token Generator	200	Successful token generation.
Update Booking	200	Successful update.
Delete Booking	201	The supplied collection asserts 201 for this demo endpoint.
GET invalid ID	404	Record not found.
Create with malformed JSON	400	Bad request in the supplied negative test.

Do not generalize demo status codes HTTP conventions provide guidance, but the endpoint contract decides the expected result. In particular, the collection expects 200 for Create Booking and 201 for Delete Booking. Preserve those expectations when reproducing this exact demo unless the live API documentation or response behavior has changed.

Combining response capture and tests

```
const responseBody = pm.response.json();

pm.test("Create booking returned an ID", function () {
    pm.response.to.have.status(200);
    pm.expect(responseBody).to.have.property("bookingid");
});

if (responseBody.bookingid) {
    pm.environment.set("b_id", String(responseBody.bookingid));
}
```

The test checks both the status and the presence of bookingid. The script then stores the ID for later requests. A useful improvement is to capture variables only when the response is successful and the expected property exists.

11. Assert the Response Body

The walkthrough uses pm.response.text() and checks whether the response text includes Josh. This is a text search across the entire response body.

```
pm.test("Verify Body matches string", function () {
    pm.expect(pm.response.text()).to.include("Josh");
});
```

This is simple, but it can be fragile: the text might appear in another field, a different guest may be returned, or the JSON property may exist with a different value. For JSON responses, parse the body and assert a specific property.

Recommended JSON property assertion

```
const body = pm.response.json();

pm.test("First name is Abhi", function () {
    pm.expect(body.firstname).to.eql("Abhi");
});
```

Compare against the environment value

```
const body = pm.response.json();
const expectedName = pm.environment.get("fname");

pm.test("First name matches the environment variable", function () {
```

```
    pm.expect(body.firstname).to.eql(expectedName);
});
```

Mismatch identified in the uploaded collection Create Booking sends firstname from {{fname}}; the walkthrough sets fname to Abhi. However, GET Booking Details As Per Id has an assertion that the response text includes Josh. Unless the actual booking being retrieved has Josh as its first name, this assertion will fail. Make the expected value match the created record, or compare against the saved fname variable.

12. Verify Response Headers and Content-Type

The collection checks the Content-Type response header and expects application/json; charset=utf-8. Content-Type describes the media type of the response body. It helps the client understand how to parse the returned data.

```
pm.test("Content-Type is present", function () {
    pm.response.to.have.header("Content-Type", "application/json; charset=utf-8");
});
```

This assertion checks an exact header value. Exact comparison can be brittle because servers may return application/json without a charset, use a different charset notation, or vary capitalization. If the goal is to confirm JSON media type, a contains check is more flexible.

```
const contentType = pm.response.headers.get("Content-Type") || "";

pm.test("Response content type is JSON", function () {
    pm.expect(contentType.toLowerCase()).to.include("application/json");
});
```

Request Content-Type vs. response Content-Type

Header	Direction	Purpose
Content-Type: application/json	Request header	Tells the server the request body is JSON.
Accept: application/json	Request header	Tells the server the client prefers a JSON response.
Content-Type: application/json	Response header	Tells the client the response body is JSON.

Collection review The exported Create Booking and Token Generator requests have raw JSON bodies but their explicit request header arrays are empty. Verify Postman supplies the appropriate header automatically in your current setup, or add Content-Type: application/json explicitly so the request configuration is clear and portable.

13. Verify Response Time

Postman exposes the response time through pm.response.responseTime, measured in milliseconds. The uploaded flow labels the test “Response time is less than 200ms” but the code checks whether the response time is below 2000.

```
pm.test("Response time is less than 2000ms", function () {
    pm.expect(pm.response.responseTime).to.be.below(2000);
});
```

Understand the threshold

Threshold	Meaning
200 ms	0.2 seconds

1000 ms	1 second
2000 ms	2 seconds

Fix the label or the number If your target is 200 milliseconds, use .below(200). If your target is 2 seconds, keep .below(2000) and name the test “less than 2000ms”. The test name and assertion should describe the same threshold.

A single slow response does not prove a performance defect. Network conditions, demo-server load, geography, and temporary service behavior can influence response time. For performance testing, define a threshold and sample size based on the application requirement, then use an appropriate load-testing approach.

14. Correct Execution Order for the Collection

The exported collection order is not ideal for a complete end-to-end run. GET Booking Details As Per Id appears before Create Booking, but its URL depends on {{b_id}}. Update Booking appears before Token Generator, but it depends on {{b_token}}. If these variables are empty or stale, the requests can fail or target the wrong record.

Recommended safe sequence

- Select the Booking API environment and confirm fname is set.
- Run Token Generator and confirm b_token is saved.
- Run Create Booking and confirm b_id is saved from the response.
- Run GET Booking Details As Per Id and verify the newly created booking.
- Run Update Booking and verify it returns the expected status.
- Run GET Booking Details As Per Id again and assert the updated values.
- Run Delete Booking only for the test booking created in this run.
- Run GET Booking Details As Per Id one final time and assert the documented not-found result.

Collection Runner A collection folder organizes requests; it does not automatically infer dependencies. The Collection Runner executes in the configured order. Reorder requests or use scripts and runner settings deliberately. Do not assume an earlier response automatically becomes a later request variable unless you explicitly save it.

Data lifecycle diagram

```
Token Generator  --->  save b_token
                  |
Create Booking    --->  save b_id
                  |
GET booking       --->  verify creation
                  |
PUT booking       --->  update using b_id + b_token
                  |
GET booking       --->  verify update
                  |
DELETE booking    --->  delete using b_id + b_token
                  |
GET booking       --->  verify deletion / not found
```

15. Real-World QA Scenarios

Scenario 1 - Booking platform

A traveller creates a hotel booking. The QA test sends POST /booking, captures the generated booking ID, retrieves the booking, updates the guest name, verifies the updated data, and deletes the test booking. This checks the resource lifecycle and the hand-off of an ID between requests.

Scenario 2 - E-commerce order

A customer places an order. Save orderId from the create-order response. Use it to retrieve order details, update an allowed field such as delivery instructions, and cancel the order when the test finishes. Confirm that the order cannot be retrieved as active after cancellation if that is the documented behavior.

Scenario 3 - Authentication and authorization

Call a protected endpoint without a token, with an invalid token, and with a valid token. Confirm that unauthorized requests are rejected and authorized requests work. Distinguish authentication (who are you?) from authorization (are you allowed to perform this operation?).

Scenario 4 - Input validation

For a registration endpoint, try a missing email, malformed email, excessively long name, wrong data type, and duplicate email. Verify the response status, error structure, and that invalid data is not created.

Scenario 5 - API regression run

Store stable requests in a collection and use separate environments for QA and staging. Run the same assertions after a deployment. Compare the result with the API contract and investigate failures instead of changing assertions simply to make the run green.

16. Best Practices

- Use a separate environment for each target system, such as Dev, QA, and Staging.
- Select and verify the active environment before running variable-based requests.
- Use variables for base URLs, IDs, names, and tokens instead of hard-coding changing values.
- Capture response values only after confirming the response is successful and the expected field exists.
- Keep the request order aligned with dependencies in a collection run.
- Use exact JSON property assertions rather than searching the entire response text when possible.
- Validate status code, response body, headers, and relevant business rules.
- Keep negative scenarios independent: one invalid condition per test where practical.
- Separate malformed JSON tests from valid JSON containing invalid field values.
- Use boundary-value tests based on documented limits; do not invent max lengths.
- Keep credentials and tokens out of public/shared collections, screenshots, source control, and logs.
- Use only test records for destructive actions such as DELETE.
- Use a cleanup strategy so test data does not accumulate.
- Avoid exact response-header matching unless the API contract requires the exact value.
- Choose response-time thresholds from non-functional requirements, not arbitrary guesses.

- Give each test a clear name that describes the expected behavior.

17. Common Problems and Troubleshooting

Problem	Likely reason	How to investigate
{{fname}} is not replaced	Wrong environment selected or variable misspelled	Select Booking API and verify fname spelling/value.
GET by ID returns 404 unexpectedly	b_id missing, stale, or wrong	Run Create Booking first and inspect b_id.
PUT/DELETE returns auth failure	b_token missing/expired or Cookie header wrong	Run Token Generator; confirm header key Cookie and value token={{b_token}}.
Body assertion fails on Josh	Expected value differs from created record	Compare response firstname with {{fname}} or set a matching expected value.
Content-Type assertion fails	Header value differs in charset/casing	Check actual response headers; assert application/json if appropriate.
Response-time test fails	Threshold too strict or server is slow	Check actual ms value and align label/threshold with requirement.
400 from Create Booking	Invalid JSON, missing header, or invalid data	Validate JSON syntax and request Content-Type; inspect error response.
Runner calls dependent request too early	Collection order does not match variable creation	Run token and create requests before dependent operations.

18. Practical Exercise

Build or refine your Booking API collection using the following tasks. Run the requests only against the demo/test service and only delete the booking created for the exercise.

- Create an environment named Booking API with fname = Abhi.
- Add a Token Generator script that saves token as b_token.
- Add a Create Booking script that validates status and bookingid, then saves b_id.
- Use {{b_id}} in GET, PUT, and DELETE URLs.
- Use Cookie: token={{b_token}} for protected operations as shown in the collection.
- Add a GET invalid-ID test that asserts 404.
- Add a malformed JSON test that asserts the documented client error.
- Add separate tests for missing fields, invalid type, and over-limit text using contract-based expectations.
- Add assertions for the expected guest name, JSON response media type, and response time.
- Run the requests in the correct order and verify the booking before and after update and deletion.

Sample combined script for Create Booking

```
const body = pm.response.json();
pm.test("Status is 200", function () { pm.response.to.have.status(200); });
pm.test("Booking ID exists", function () {
    pm.expect(body).to.have.property("bookingid");
    pm.expect(body.bookingid).to.be.a("number");
});
pm.test("First name matches fname", function () {
    pm.expect(body.booking.firstname).to.eql(pm.environment.get("fname"));
});
if (body.bookingid) pm.environment.set("b_id", String(body.bookingid));
```

19. Interview Revision Questions

- What is negative API testing? Give three examples.
- When should an API return 404 Not Found?
- What is the difference between malformed JSON and a valid JSON body with an invalid field value?
- What is a Postman environment variable? How do you reference it in a request?
- What does pm.response.json() do?
- What is the difference between pm.environment.set() and pm.environment.get()?
- Why should a generated booking ID be stored in a variable?
- How do you pass a token stored in b_token to the Booking API?
- What does pm.test() do?
- How do you assert a response status code and a JSON property?
- Why can exact Content-Type header comparisons become brittle?
- Why must collection execution order match request dependencies?
- Why is a 200 response alone not enough to prove an API test passed?
- How would you test a maximum field length if the limit is specified as 50 characters?

20. Key Takeaways and Source Coverage

- Negative scenarios check that the API rejects missing, invalid, malformed, or unavailable data correctly.
- Environment variables keep requests reusable and environment-specific.
- pm.response.json() reads JSON response data; pm.environment.set() stores a value for later requests.
- The booking ID and token must be captured before dependent GET, PUT, and DELETE requests run.
- Postman assertions should match the API contract and validate meaningful response data.
- The supplied collection contains a few inconsistencies that should be corrected before relying on it for automated regression.

Source coverage: Flow(5).txt supplies the negative scenarios, environment variable walkthrough, b_id and b_token capture scripts, status/body/header/response-time assertions, and the 404/400 examples. Booking API.postman_collection.json supplies the request names, URLs, request bodies, headers, and embedded test scripts used in the collection review. Additional explanations and recommended scripts in these notes are teaching enhancements based on those uploaded examples.

Final learning sequence First practise environment variables and response chaining. Next learn status-code and JSON-body assertions. Then add header and response-time checks. Finally, run the complete create → read → update → read → delete lifecycle, followed by independent negative tests.